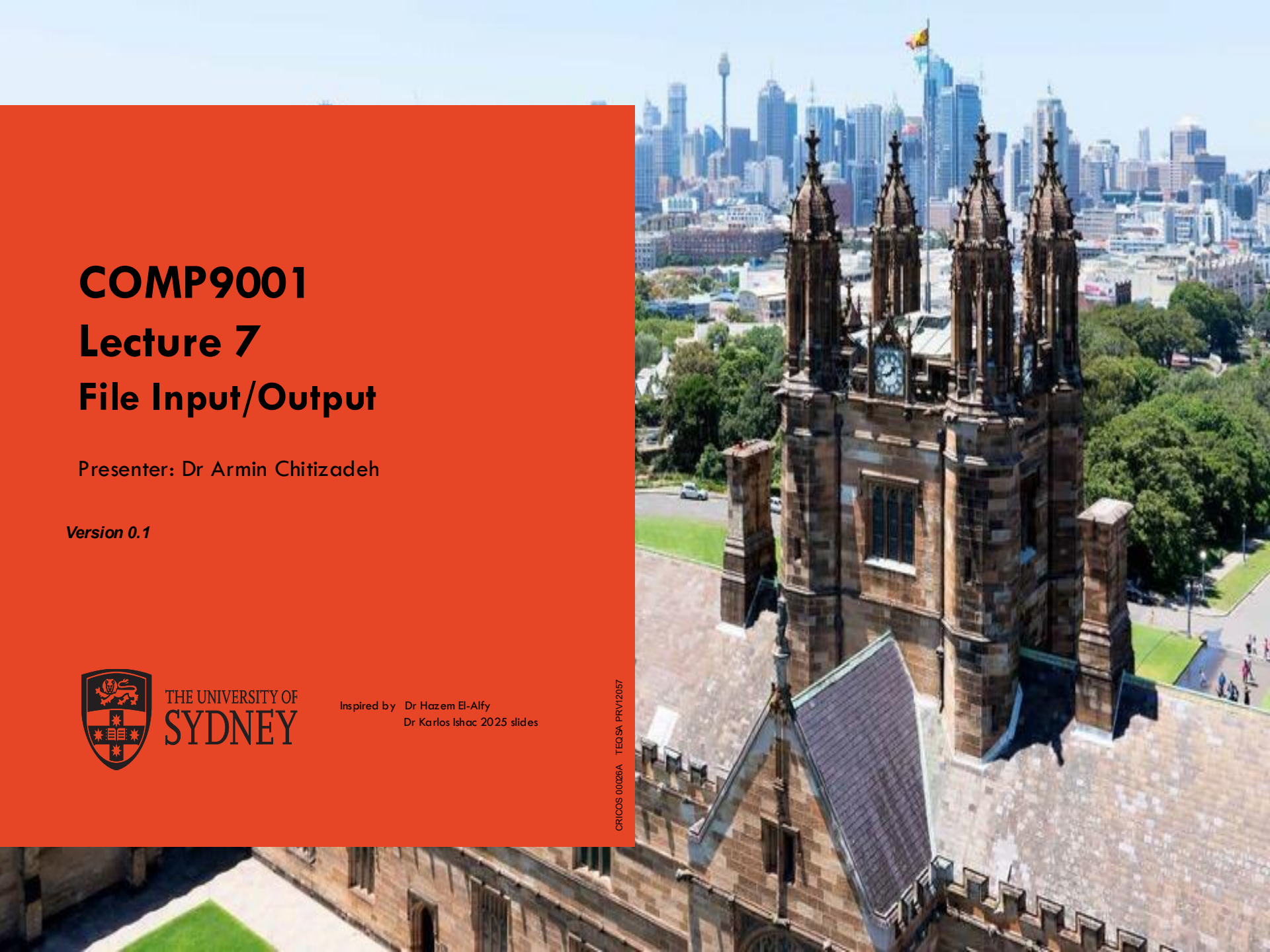
COMP9001
Lecture 7
File Input/Output

Presenter: Dr Armin Chitizadeh

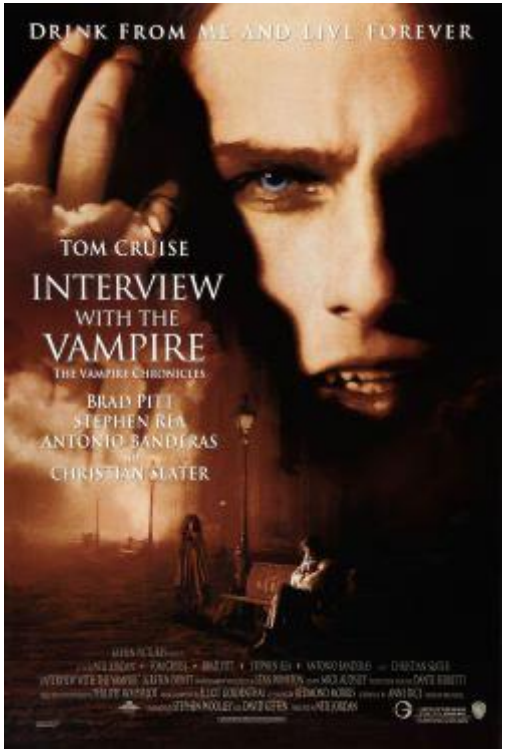
Version 0.1



THE UNIVERSITY OF SYDNEY

Inspired by Dr Hazem El-Alfy
Dr Karlos Ishac 2025 slides

Assignment1 interview



Final Project

- https://www.menti.com/alwmng15xeqo

Where are we in the unit?

Input/Output

- Inputs:
  - Standard input – `input()`
  - Command line arguments
  - File inputs – read from file
- Output:
  - Standard output – `print()`
  - File output – write to file

```
1  import sys
2
3  # Write comments here
4  N = len(sys.argv)
5  i = 0
6  while i < N:
7      print("Inputs from the command line:")
8      print(sys.argv[i])
9      i += 1
10
11 # Write comments here
12 print("Inputs from standard input:")
13 print(input())
```

Your task:

- Go to Ed Lesson. Review the code in `demo.py`.
- Write comments for each code block to explain the purpose of each code block.

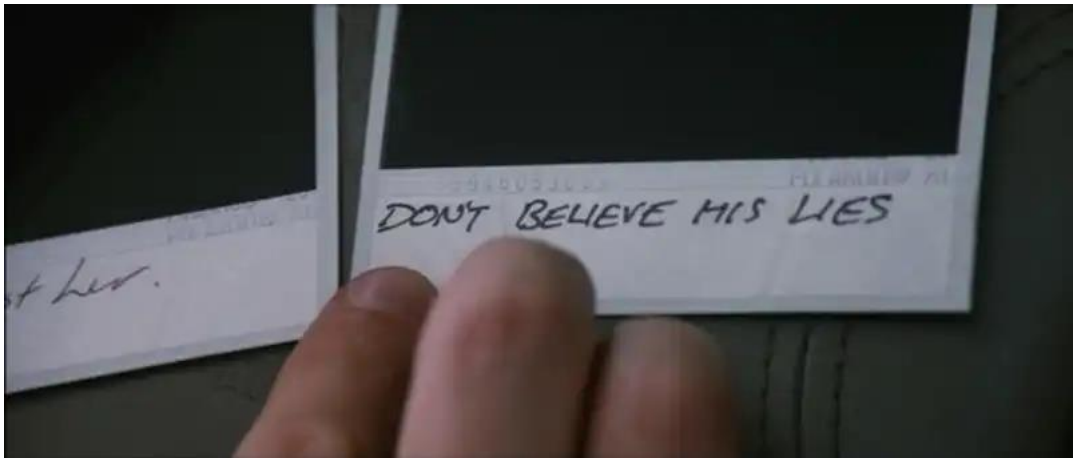
What is your **favourite** computer game?

- https://www.menti.com/al27bgdhaaxn

Memento


MEMENTO


THE FACTS:
FACT 1: MALE
FACT 2: WHITE


DON'T BELIEVE HIS LIES

Why is file input/output important?

- Programs that we have written runs for a short time, uses data supplied by users from the terminal and produces transient data.

- Programs you write in the future may:
  - Run *indefinitely*.
  - *Produce* data that you will need to capture at different points of time to be used by others, sometime in the future.
  - *Use* data that is stored in multiple files.
  - Read and write data larger than what can fit in memory

Read and Write IRL

Phases	Read	Write
Before	• Locate book. • Check book details – name, size, colour/author? • Permission to access book.	• Locate book. • Check book details – name, size, colour? • Permission to access book.
During	• Open Book. • Read [all/specified] contents from [start/page] to [end/page]	• Open Book. • Write [all/specified] contents to [start/location/end] of page
After	Close Book	Close Book

Your task:

- Did you notice anything in common between both read and write tasks?
- Highlight the common steps.

General Process of File I/O

1. Create file object with the correct mode (access permission).
   - `open(file, mode)`
   - https://docs.python.org/3/library/functions.html#open

2. Call file object *methods* to read or write to file.
   - `read(size)`
   - `readline()`
   - `readlines()`
   - `write(text)`
   - https://docs.python.org/3/library/io.html#module-io

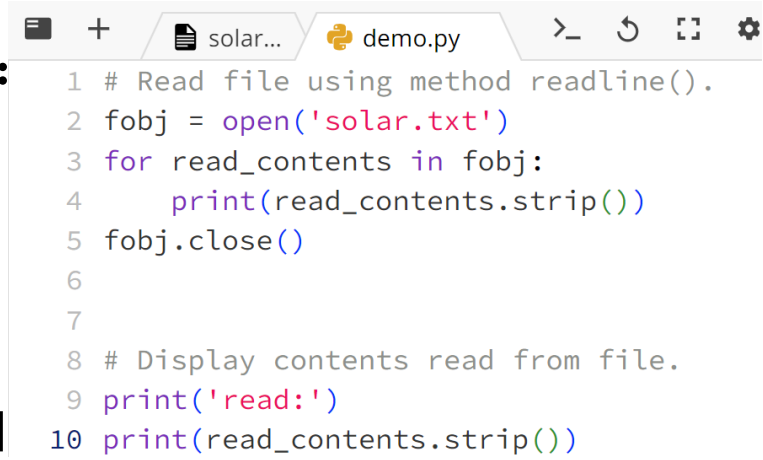
3. Close file using file object method `close()`.

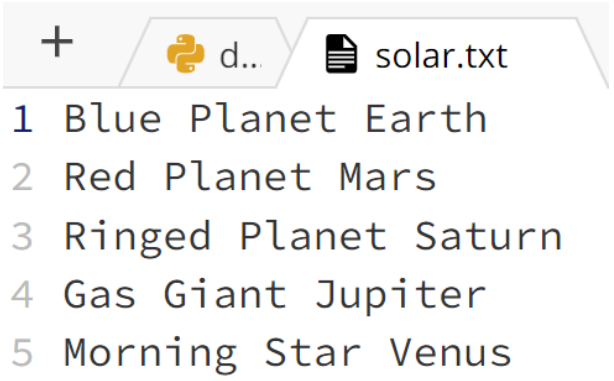
File Input: `read()`, `readline()`, `readlines()`

- Can you identify the line numbers that:
  - Create a file object in read mode?
  - Read the first line of the file?
  - Close the file object?

- Go to Ed Lesson. Run the program and observe the output.

- Determine the output of the program if you modify Line 3 to:
  - `read_contents = fobj.read()?`
  - `read_contents = fobj.readlines()?`





File Output: `write()`, `print()`

- Go to Ed Lesson. Review the codes in `demo.py`.

```
1 # Write to file
2 filename = "shopping.txt"
3 fobj = open(filename, "w")
4 char = fobj.write("Coffee\n")
5 print("Milk", file=fobj)
6 fobj.close()
7
8 # Display total characters written by...abs
9 print(f"We have written {char} to {filename}!")
```

- Can you identify the line numbers that:
  - Create a file object in write mode?
  - Write a given string to the file?
  - Close the file object?

- Run the program. Open the file and check the output in the terminal. Explain your observations.

Summary: Input/Output

- Inputs:
  - Standard input – `input()`
  - Command line arguments
  - File inputs – read from text file
    - `read()-> str`
    - `readline() -> str`
    - `readlines() -> list`
- Output:
  - Standard output – `print()`
  - File output – write to text file
    - `write() -> int`
    - `writelines() -> None`

Menti!

https://www.menti.com/alrmngn9t2ss

Break

Recap: File Input/Output

- Three steps to read from a file or write to a file:
  1. Create a file object in read (default) or write mode
  2. Perform desired actions by calling the file object **methods**:
     - Read data from file – `read()`, `readline()`, `readlines()`
     - Write data to file – `write()`, `writelines()`
  3. Close file:
     - `close()`

Exercise: Shopping List

- Read the question and extract information about the IPO of the program.
  - Inputs
  - Process
  - Outputs

Files – Information Storage

- Files can store information – text, images, binary data.

- Unix/linux tool called `file` that can scan the contents of a file and determine its type. Example:
  - `file hello.py`
  - `file turingprof.gif`

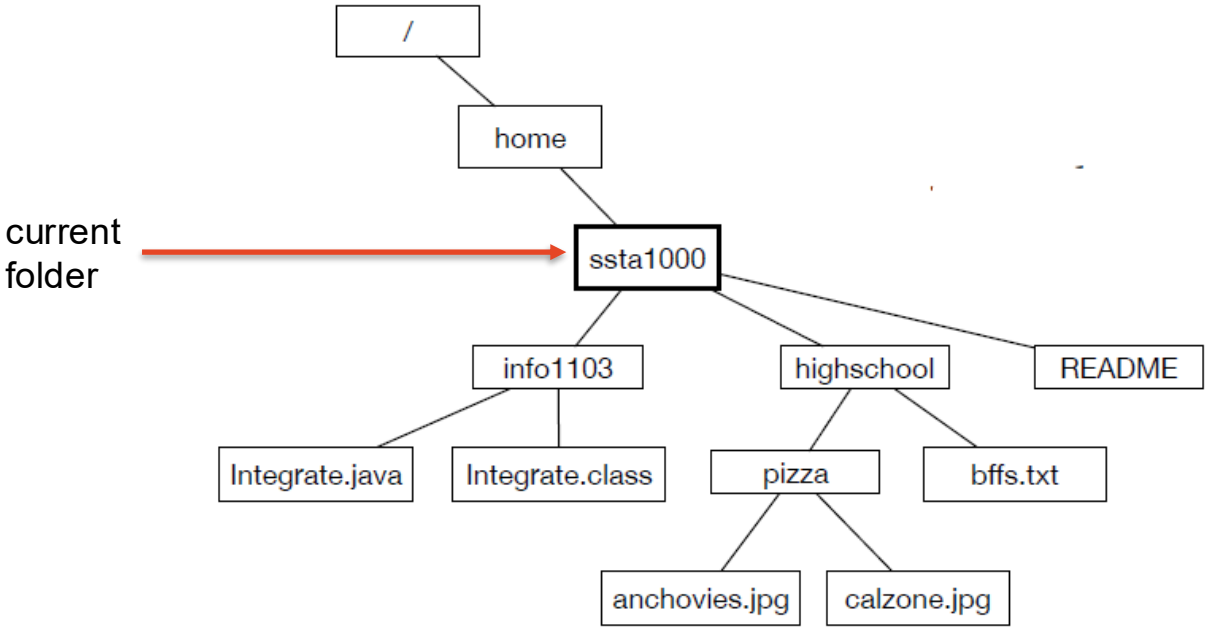
- Go to Ed Lesson and do Part 1.
  - Identify examples of files that are:
    - ASCII text?
    - Python script?
    - gif image
  - How do you inspect the file `krusty.py` inside folder `a1`?

Files – Location

- Files are organized into directories (also known as folders).
- Working directory – where you are currently at! `'pwd'`
- A string like `'/home'` that identifies a file or directory is called a **path**.
- Two types of paths:
  - **Relative** path relates to the current directory. Example:
    - Referring to file in folder: `a1/krusty.py`
    - Referring to file in same path: `hello.py`
  - **Absolute** path does not depend on the current directory and begins with `/`. Example:
    - Referring to a folder: `/home/a1`
    - Referring to a file: `/home/hello.py`

Exercise – Relative and Absolute Path

- State where is `calzone.jpg` using:
  - relative path.
  - absolute path.



os module

- The `os` module provides functions for working with files and directories.
  - Where am I? `os.getcwd()`
  - Get a list of files and folders in path: `os.listdir(path)`

- Go to Ed Lesson and do Part 2.

- Files/folders IRL. Many things can go wrong:
  - Path given by user does not exist
  - No such file or directory

- What will happen if your program does not specify what to do in these cases?

File Input without Exception Handling

- Write a program to read the number stored in the file called `numbers.txt`. This file should contain only one integer. File path is provided as a command line argument.

read_int_bad.py

```
1 import sys
2
3 the_file = open(sys.argv[1], 'r')
4 the_integer = int(the_file.readline())
5 the_file.close()
6
7 print(f'The number is: {the_integer}')
8 # End of program
```

- Anything that can go wrong will go wrong. List all the possible exceptions that can happen during the execution of this program IRL.
- Go to Ed Lesson and run the program for each given case.

try-except-finally Block

- `try` a piece of code that contains a potential rare error
- If the rare error occurs a new exception is generated (`raise`)
- The exception is caught and handled using `except`
- The optional `finally` block is always executed at the end of the except region, regardless of the try block raising an error or not :
  - After the `try` block completes successfully,
  - Or after an `except` block is executed.

```
1  try:
2      # do something
3      # which could throw
4      # Exceptions
5  except EOFError:
6      # do something to deal
7      # with the situation
8  except FileNotFoundError:
9      # do something else
10 except Exception:
11     # do something general
12 finally:
13     # code that should happen
14     # after normal/weird case
```

Concrete Exceptions

- Good practice to structure except blocks with increasing generality.

- These are the exceptions that are usually raised:
  - https://docs.python.org/3/library/exceptions.html#concrete-exceptions
  - Common exceptions:
    - `TypeError e.g. 2 + "2"`
    - `ValueError  e.g. int("a")`
    - `ZeroDivisionError e.g. 1/0`
    - `IndexError e.g. sys.argv[99999]`
    - `SyntaxError e.g. print("Hello)`
    - `NameError e.g. print(hello)`

File Input with Exception Handling

- Go to Ed Lesson and run the program for each given case.
- Did you catch all the exceptions?

Exercise: Text Files IRL

- File contains many different parts but may also contain contents that you do not expect.

- Before you can process the contents in your program, the file contents must be loaded into memory for the program to do useful work.

- Example: 2D point locations.

Exercise: 2D Points Locations

- Write a program to extract all the 2D point data e.g. (x, y) of exactly 20 locations from a given file.

- The files:
  - `points_perfect.txt`
  - `points_missing.txt`
  - `points_horror.txt`
  - `points_non_numbers.txt`

- The plan:
  1. Read each line and load into memory.
  2. Check contents of each line:
     - If valid 2D points, store as a tuple e.g. (x, y).
     - If not valid, skip.

Exercise: 2D Points Locations (Step 1)

Review the codes and answer these questions:

1. What is the data type of these variables:
   - `line`
   - `points`
   - `infile`

2. Call function `read_points()` with argument `points_perfect`. What will happen?

3. Which variable contains the contents read from file? Trace this variable during the execution of the program.

```
3  def myerr(s):
4      sys.stderr.write(s)
5
6  def read_points(infilename):
7      points = [None]*20
8      location = 0
9      line_num = 0
10     try:
11         infile = open(infilename , 'r')
12
13         done = False
14         while not done:
15
16             line = infile.readline()
17             if not line:
18                 break
19             line_num += 1
20
21             # ???
22
23         infile.close()
24     except FileNotFoundError as fnfe:
25         myerr("file {} not found\n".format(infilename))
26
27 # Author: Dr. John Stavrakakis
```

Exercise: 2D Points Locations (Step 2)

Review the codes and answer these questions:

1. What are the values in `tokens` if `line` takes on these values:
   - `"happy camper"`
   - `"12, 40.5"`
   - `"35, lol"`
   - `"10, 32, 55"`

2. Identify which `tokens` in Q1 will throw `ValueError` exception?

PointFileReader_pa...

```
1  # extract tokens from one line
2  tokens = line.split(',')
3  if len(tokens) < 2:
4      # bad line, skip to next line
5      myerr("less than 2 tokens on line #{}\n".format(line_num))
6      continue
7
8  # parse integers from 1st and 2nd tokens
9  try:
10     x = int(tokens[0].strip())
11     y = int(tokens[1].strip())
12
13     # create a new point with data
14     pair = (x,y)
15     # update the array
16     points[location] = pair
17     location = location + 1
18
19 except ValueError as ve:
20     # bad number, skip to next line
21     myerr("could not convert to int on line #{}\n".format(line_num))
22     continue
23 # Author: Dr. John Stavrakakis
```

Exercise: 2D Points Locations (PIAT)

- Execute the file `PointFileReader.py` using each of the given files.

- Observe the output.

Reading This Week

- **Chapter 9 and 14.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.

- **Chapter 13.2, 13.6 Liang, Y. (2022)**. Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.

- **Python Tutorial:**

- https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files
- https://docs.python.org/3/tutorial/errors.html

Reading This Week

- **Chapter 14 and 9.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.

- **Chapter 13.2, 13.6** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.

- Python Tutorial:
  - https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files
  - https://docs.python.org/3/tutorial/errors.html

Discovery Activity: Human Digitizing

Access the Ed Activity here.

You are tasked with investigating experimental human digitizing techniques and recall your knowledge in File I/O from COMP9001. Use this to perform the following actions:

(a) Read in the file 'human.txt' and print each line.

(b) Create a new file based on 'human.txt' named 'human_clone.txt'

(c) Copy over all details but modify the 'name' to have the word 'Two' added to the end.

(d) Also half the original age of the human.

(e) Now append the detail 'Eye Colour: Blue'.

(f) Verify your program ran correctly by checking your output file.